deleteActivity error Error: "<Activity: test>" needs to have a value for field "id" before this many-to-many relationship can be used.
    at postGraphQL (page.tsx:72:26)
    at async deleteActivityFromStructure (page.tsx:1613:7)
deleteActivityFromStructure @ page.tsx:1624
await in deleteActivityFromStructure
onConfirm @ page.tsx:2206
onConfirm @ page.tsx:3115
i see this error when i try to delete activity , on firt click i see this error but when i try to delete it again it deletes succesfully

```
from leaderboard.decorators import graphql_permission_required
import graphene
from django.utils.dateparse import parse_datetime
from django.contrib.auth import get_user_model
from ...models import Program, Activity,ACTIVITY_TYPE_CHOICES
from attachments.models import SecureAttachment
from ..types import ProgramActivityType
from leaderboard.events import send_subscription_and_notification
from django.db.models import Q
from django.db.models import Prefetch

User = get_user_model()

class CreateActivity(graphene.Mutation):
    class Arguments:
        type = graphene.String(required=True)
```

```
        name = graphene.String(required = True)
        template_id = graphene.Int()
        description = graphene.String()
        deadline = graphene.String()
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        mapp_tool = graphene.String()
        is_feedback = graphene.Boolean()
        attachment_ids = graphene.List(graphene.Int)

    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, type,
name,template_id=None,description=None, deadline=None,
mapp_survey=None, is_360=None,mapp_survey_name = None,
is_feedback=False,mapp_tool = None,attachment_ids=None):
        try:
            deadline_parsed = parse_datetime(deadline) if deadline else
None

            activity = Activity(
                type=type,
                name = name,
                template_id = template_id,
```

```
            description=description if description else '',
            deadline=deadline_parsed,
            mapp_survey=mapp_survey,
            is_360=is_360,
            mapp_survey_name = mapp_survey_name,
            is_feedback = is_feedback,
            mapp_tool = mapp_tool,
        )
        activity.full_clean()
        activity.save()
        if attachment_ids:
            attachments = SecureAttachment.objects.filter(
                id__in=attachment_ids,
                is_active=True
            )
            activity.attachments.set(attachments)

        send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
            subscription_class="ActivitySubscription",
            group=info.context.user.username,
            subscription_payload={
                "object": activity,
                "mutation": "CREATE",
            },
            # notification_event_type="survey_token_created",
            # notification_payload={
            #     "object": token,
```

```
            #     "user_id": user.id,
            # },
        )
        return CreateActivity(activity=activity, success=True,
message="Activity created successfully")
    except Program.DoesNotExist:
        return CreateActivity(activity=None, success=False,
message="Program not found.")
    except Exception as e:
        return CreateActivity(activity=None, success=False,
message=str(e))


class UpdateActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
        template_id = graphene.Int()
        name = graphene.String()
        type = graphene.String()
        description = graphene.String()
        deadline = graphene.String()  # ISO 8601 datetime string
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        is_feedback = graphene.Boolean()
        mapp_tool = graphene.String()
        attachment_ids = graphene.List(graphene.Int)

    activity = graphene.Field(ProgramActivityType)
```

```
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id, template_id=None,name=None,
type=None, description=None, deadline=None, mapp_survey=None,
is_360=None,mapp_survey_name = None,
is_feedback=None,mapp_tool = None,attachment_ids=None):
        try:
            activity = Activity.objects.get(pk=id)

            # Handle type with choices validation
            if type is not None:
                activity.type = type

            # Handle deadline parsing
            if deadline is not None:
                deadline_parsed = parse_datetime(deadline)
                if not deadline_parsed:
                    return UpdateActivity(activity=None, success=False,
message="Invalid deadline format.")
                activity.deadline = deadline_parsed

            # Update simple fields (None will set NULL)
            if name is not None:
                activity.name = name
            if description is not None:
                activity.description = description
```

```
            activity.mapp_survey = mapp_survey
            activity.mapp_survey_name = mapp_survey_name
            activity.mapp_tool = mapp_tool
            activity.is_360 = is_360
            if template_id:
                activity.template_id = template_id

            if is_feedback is not None:
                activity.is_feedback = is_feedback
            activity.full_clean()
            activity.save()
            if attachment_ids is not None:
                attachments = SecureAttachment.objects.filter(
                    id__in=attachment_ids,
                    is_active=True
                )
                activity.attachments.set(attachments)

            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "UPDATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
```

```
            #     "object": token,
            #     "user_id": user.id,
            # },
        )

        participants = User.objects.filter(
            program_enrollments__program__in=Program.objects.filter(
                Q(pre_activity=activity) |
                Q(post_activity=activity) |
                Q(sessions__pre_activity=activity) |
                Q(sessions__post_activity=activity) |
                Q(sessions__topic__pre_activity=activity) |
                Q(sessions__topic__post_activity=activity)
            )
        ).distinct()

        for participant in participants:
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=participant.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "UPDATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
```

```
                #     "user_id": user.id,
                # },
            )
        return UpdateActivity(activity=activity, success=True,
message="Activity updated successfully")
    except Activity.DoesNotExist:
        return UpdateActivity(activity=None, success=False,
message="Activity not found.")
    except Exception as e:
        return UpdateActivity(activity=None, success=False,
message=str(e))


class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)
            activity.delete()
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
```

```
            subscription_class="ActivitySubscription",
            group=info.context.user.username,
            subscription_payload={
                "object": activity,
                "mutation": "DELETE",
            },
            # notification_event_type="survey_token_created",
            # notification_payload={
            #     "object": token,
            #     "user_id": user.id,
            # },
        )
        return DeleteActivity(success=True, message="Activity deleted
successfully")
    except Activity.DoesNotExist:
        return DeleteActivity(success=False, message="Activity not
found")


class Mutation(graphene.ObjectType):
    create_activity = CreateActivity.Field()
    update_activity = UpdateActivity.Field()
    delete_activity = DeleteActivity.Field()

"""
Example:
mutation {
  createActivity(
    type: "quiz"
```

```
    description: "A quiz for the program"
    deadline: "2025-12-10T17:00:00Z"
    mappSurvey: 123
    is360: true
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360

    }
  }
}

mutation {
  updateActivity(
    id: "5"
    type: "audio"
    description: "Updated description"
    deadline: "2025-12-15T10:30:00Z"
    mappSurvey: 456
    is360: false
  ) {
```

```
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360
    }
  }
}

mutation {
  deleteActivity(id: "5") {
    success
    message
  }
}
""" this is my program/graphql/activity/mutations.py
```

The issue is in your `DeleteActivity` **mutation**.

✅ Why this happens

This line is the real problem:

```
activity.delete()

send_subscription_and_notification(
    ...
    subscription_payload={
        "object": activity,
        "mutation": "DELETE",
    },
)
```

After `activity.delete()`, Django clears the instance's PK (`id = None`).

So when your subscription / GraphQL serializer later tries to access:

- `activity.id`
- or worse, `activity.attachments` (many-to-many)

Django throws:

> `"<Activity: test>" needs to have a value for field "id" before this many-to-many relationship can be used.`

That is exactly why:

- **1st click** → backend deletes the row, then crashes while trying to serialize/send deleted object
- **2nd click** → row is already gone, frontend refresh/state changes, so it *looks* deleted

---

✅ Fix (best practice)

You must collect everything **before delete**, and **do not send the deleted model instance** directly after deletion.

✅ Proper fix for `program/graphql/activity/mutations.py`

Path

`program/graphql/activity/mutations.py`

🔧 Replace your current `DeleteActivity` class with this

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)

            # IMPORTANT: collect everything BEFORE delete
            activity_id = activity.id
            activity_name = activity.name
            activity_type = activity.type
```

```
            activity_description = activity.description
            activity_deadline = activity.deadline
            activity_mapp_survey = activity.mapp_survey
            activity_is_360 = activity.is_360
```

✅ Why this fixes it

Because now:

- you fetch activity **with attachments before delete**
- store all needed values in normal Python variables
- delete the DB row
- send a **plain dict** instead of deleted ORM object

So no code later can accidentally do:

Python Run

on a deleted object.

⚠️ Very important: your subscription code may still expect a model instance

If your subscription code or `send_subscription_and_notification` expects:

Python Run

instead of:

Python Run

then this dict approach may need adjustment.

If your subscription strictly expects an ORM instance:

Then the safer approach is:

- send notification **before** delete, OR
- create a lightweight fake payload object

But **for DELETE events**, best practice is usually:

- send only `id` + minimal metadata
- don't send full ORM object

✅ Safer minimal version (recommended if subscription only needs ID)

If your frontend only needs deleted ID to remove item from UI, use this instead:

```python
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
```

```python
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)

            activity_id = activity.id
            activity_name = activity.name

            activity.delete()

            send_subscription_and_notification(
                subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
```

🔥 Most likely exact root cause in your project

Because your `ProgramActivityType` probably includes:

- `attachments`
- or related fields

and your subscription is probably serializing the object through Graphene.

After delete:

Python Run

Then when Graphene resolves:

```python
```

Python Run

Django raises:

> needs to have a value for field "id" before this many-to-many relationship can be used

---

✅ Step-by-step change with proper path

File path

`program/graphql/activity/mutations.py`

Step 1 — Find your current DeleteActivity

Search for:

```python
```

Python Run

Step 2 — Replace this block

Current code

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)
            activity.delete()
            send_subscription_and_notification(
                subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
                },
            )
```

Step 3 — Paste this fixed version

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
```

```
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)

            # IMPORTANT: collect everything needed BEFORE delete
            activity_id = activity.id
            activity_name = activity.name
            activity_type = activity.type
            activity_description = activity.description
            activity_deadline = activity.deadline
            activity_mapp_survey = activity.mapp_survey
            activity_is_360 = activity.is_360
            activity_mapp_survey_name = activity.mapp_survey_name
```

✅ Also do this small improvement in `CreateActivity`

You have:

```
except Program.DoesNotExist:
```

Python Run

But `CreateActivity` does **not** fetch `Program` at all.

So this block is unnecessary / misleading.

Replace:

```python
except Program.DoesNotExist:
    return CreateActivity(activity=None, success=False, message="Program not found.")
```

Python Run

With:

```python
```

Python Run

✅ Also do this small improvement in `DeleteActivity`

Right now your old code has no generic exception handler.

Always add:

```python
except Exception as e:
```

Python Run

So frontend gets actual error.

🚨 If error still comes after this

Then the problem is in:

Possible path

`program/graphql/Activity/subscriptions.py`

or inside:

Possible path

`leaderboard/events.py`

Specifically, if DELETE subscription resolver is still trying to treat `payload["object"]` as a Django model.

Example bad code:

Python Run

It must be changed to:

Python Run

or for delete event just:

Python Run

✅ My recommendation (best architecture)

For **DELETE subscriptions**, only send:

```
{
  "object": {"id": activity_id},
  "mutation": "DELETE"
}
```

Frontend only needs the ID to remove it from state.

This avoids all serializer issues.

✅ Final root cause in one line

You are sending a deleted Django model instance (`activity`) into subscription code after `activity.delete()`, and Graphene/Django tries to access its many-to-many `attachments`, which fails because deleted objects no longer have a valid `id`.

If you want, send me these 2 files next and I'll give you **exact final production-safe code** (with proper paths, line-by-line changes like before):

1. `program/graphql/Activity/subscriptions.py`
2. `leaderboard/events.py`

That will let me give you the **100% correct final delete fix** for your project structure.

```
from leaderboard.decorators import graphql_permission_required
import graphene
from django.utils.dateparse import parse_datetime
from django.contrib.auth import get_user_model
from ...models import Program, Activity,ACTIVITY_TYPE_CHOICES
from attachments.models import SecureAttachment
from ..types import ProgramActivityType
from leaderboard.events import send_subscription_and_notification
from django.db.models import Q
from django.db.models import Prefetch

User = get_user_model()

class CreateActivity(graphene.Mutation):
    class Arguments:
        type = graphene.String(required=True)
        name = graphene.String(required = True)
        template_id = graphene.Int()
        description = graphene.String()
        deadline = graphene.String()
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        mapp_tool = graphene.String()
        is_feedback = graphene.Boolean()
        attachment_ids = graphene.List(graphene.Int)
```

```
    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, type,
name,template_id=None,description=None, deadline=None,
mapp_survey=None, is_360=None,mapp_survey_name = None,
is_feedback=False,mapp_tool = None,attachment_ids=None):
        try:
            deadline_parsed = parse_datetime(deadline) if deadline else
None

            activity = Activity(
                type=type,
                name = name,
                template_id = template_id,
                description=description if description else '',
                deadline=deadline_parsed,
                mapp_survey=mapp_survey,
                is_360=is_360,
                mapp_survey_name = mapp_survey_name,
                is_feedback = is_feedback,
                mapp_tool = mapp_tool,
            )
            activity.full_clean()
            activity.save()
```

```
            if attachment_ids:
                attachments = SecureAttachment.objects.filter(
                    id__in=attachment_ids,
                    is_active=True
                )
                activity.attachments.set(attachments)

            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "CREATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
            return CreateActivity(activity=activity, success=True,
message="Activity created successfully")
        except Program.DoesNotExist:
            return CreateActivity(activity=None, success=False,
message="Program not found.")
        except Exception as e:
            return CreateActivity(activity=None, success=False,
```

```
message=str(e))


class UpdateActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
        template_id = graphene.Int()
        name = graphene.String()
        type = graphene.String()
        description = graphene.String()
        deadline = graphene.String()  # ISO 8601 datetime string
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        is_feedback = graphene.Boolean()
        mapp_tool = graphene.String()
        attachment_ids = graphene.List(graphene.Int)

    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id, template_id=None,name=None,
type=None, description=None, deadline=None, mapp_survey=None,
is_360=None,mapp_survey_name = None,
is_feedback=None,mapp_tool = None,attachment_ids=None):
        try:
```

```
        activity = Activity.objects.get(pk=id)

        # Handle type with choices validation
        if type is not None:
            activity.type = type

        # Handle deadline parsing
        if deadline is not None:
            deadline_parsed = parse_datetime(deadline)
            if not deadline_parsed:
                return UpdateActivity(activity=None, success=False,
message="Invalid deadline format.")
            activity.deadline = deadline_parsed

        # Update simple fields (None will set NULL)
        if name is not None:
            activity.name = name
        if description is not None:
            activity.description = description
        activity.mapp_survey = mapp_survey
        activity.mapp_survey_name = mapp_survey_name
        activity.mapp_tool = mapp_tool
        activity.is_360 = is_360
        if template_id:
            activity.template_id = template_id

        if is_feedback is not None:
            activity.is_feedback = is_feedback
        activity.full_clean()
```

```
        activity.save()
        if attachment_ids is not None:
            attachments = SecureAttachment.objects.filter(
                id__in=attachment_ids,
                is_active=True
            )
            activity.attachments.set(attachments)

        send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
            subscription_class="ActivitySubscription",
            group=info.context.user.username,
            subscription_payload={
                "object": activity,
                "mutation": "UPDATE",
            },
            # notification_event_type="survey_token_created",
            # notification_payload={
            #     "object": token,
            #     "user_id": user.id,
            # },
        )

        participants = User.objects.filter(
            program_enrollments__program__in=Program.objects.filter(
                Q(pre_activity=activity) |
                Q(post_activity=activity) |
                Q(sessions__pre_activity=activity) |
```

```
                Q(sessions__post_activity=activity) |
                Q(sessions__topic__pre_activity=activity) |
                Q(sessions__topic__post_activity=activity)
            )
        ).distinct()

        for participant in participants:
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=participant.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "UPDATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
        return UpdateActivity(activity=activity, success=True,
message="Activity updated successfully")
    except Activity.DoesNotExist:
        return UpdateActivity(activity=None, success=False,
message="Activity not found.")
    except Exception as e:
        return UpdateActivity(activity=None, success=False,
```

```
message=str(e))


class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)
            activity_id = activity.id
            activity_name = activity.name
            activity.delete()
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": {
                        "id": activity_id,
                        "name": activity_name,
                    },
                    "mutation": "DELETE",
```

```
            },
        )
        return DeleteActivity(success=True, message="Activity deleted
successfully")
    except Activity.DoesNotExist:
        return DeleteActivity(success=False, message="Activity not
found.")
    except Exception as e:
        return DeleteActivity(success=False, message=str(e))


class Mutation(graphene.ObjectType):
    create_activity = CreateActivity.Field()
    update_activity = UpdateActivity.Field()
    delete_activity = DeleteActivity.Field()

"""
Example:
mutation {
  createActivity(
    type: "quiz"
    description: "A quiz for the program"
    deadline: "2025-12-10T17:00:00Z"
    mappSurvey: 123
    is360: true
  ) {
    success
    message
    activity {
```

```
      id

      type
      description
      deadline
      mappSurvey
      is360

    }
  }
}

mutation {
  updateActivity(
    id: "5"
    type: "audio"
    description: "Updated description"
    deadline: "2025-12-15T10:30:00Z"
    mappSurvey: 456
    is360: false
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
```

```
      mappSurvey
      is360
    }
  }
}

mutation {
  deleteActivity(id: "5") {
    success
    message
  }
}
"""
```

this is working well can i keep this

✅ Yes — you can keep this

Your current `DeleteActivity` **fix is good and safe** for the delete issue you were getting.

This part is now correct:

```
activity = Activity.objects.get(pk=id)
activity_id = activity.id
activity_name = activity.name
activity.delete()

send_subscription_and_notification(
    ...
    subscription_payload={
```

```
        "object": {
            "id": activity_id,
            "name": activity_name,
        },
        "mutation": "DELETE",
    },
)
```

✅ Why it works

Because now you are:

- saving `id` **before delete**
- deleting the DB object
- sending a **plain dict**, not the deleted Django instance

So Django/Graphene will **not** try to do:

- `activity.id` after delete
- `activity.attachments.all()` after delete

That is exactly what was causing the first-click error.

✅ Short answer

Yes, keep this `DeleteActivity` code.

It is the correct fix **as long as your subscription code can handle DELETE payload as a dict** like:

```
{
  "object": {
    "id": activity_id,
```

```
    "name": activity_name,
  },
  "mutation": "DELETE",
}
```

Since you said “**this is working well**”, that means your subscription side is already handling it fine.
So:

> ✅ **Yes — keep it.**

⚠️ But make these 2 small improvements before finalizing

Your delete is fine, but I recommend **2 cleanup fixes** in the same file.

---

1) Remove wrong `except Program.DoesNotExist` in `CreateActivity`

File path

`program/graphql/activity/mutations.py`

❌ Current

```python
except Program.DoesNotExist:
    return CreateActivity(activity=None, success=False, message="Program not found.")
except Exception as e:
```

✅ Replace with

```python
except Exception as e:
```

Python Run

Why?

Because inside `CreateActivity`, you are **not calling**:

```python
```

Python Run

So `Program.DoesNotExist` will never be raised there.

---

2) Small improvement in `UpdateActivity`

This line:

```python
if template_id:
```

Python Run

can fail when `template_id = 0` or if you want to explicitly clear it.

✅ Better version

Replace:

```python
if template_id:
```

Python Run

With:

```python
if template_id is not None:
```

Python Run

✅ Final recommended version for only the changed parts

Path

`program/graphql/activity/mutations.py`

Change 1 — CreateActivity

Replace:

```python
except Program.DoesNotExist:
    return CreateActivity(activity=None, success=False, message="Program not found.")
except Exception as e:
```

Python Run

With:

```python
except Exception as e:
```

Python Run

Change 2 — UpdateActivity

Replace:

```python
if template_id:
```

Python Run

With:

```python
if template_id is not None:
```

Python Run

✅ Final verdict

DeleteActivity

✔ **Keep it exactly as you have now**

Overall file

✔️ Working
⚠️ Just make the 2 small cleanup fixes above

✅ Best practice note (important)

Since your `DELETE` payload now sends:

```
"object": {
    "id": activity_id,
    "name": activity_name,
```

Python

make sure **frontend subscription handler** does not assume DELETE event has full `ProgramActivityType`.

For delete, frontend should ideally do:

```
if (mutation === "DELETE") {
  const deletedId = payload.object.id;
  // remove from local state
```

TypeScript

and **not** expect:

- attachments
- type
- description
- etc.

Since you said it's already working, you're likely fine.

✅ My recommendation

Yes, keep this code.
It is the right fix for the error you had.

If you want, next I can give you the **final cleaned production-ready full file** for:

`program/graphql/activity/mutations.py`

with:

- your current working delete fix
- the 2 cleanup fixes
- a couple of small backend improvements
- **full copy-paste ready code** (exact final version).